ವಿಶ್ವೇಶ್ವರಯ್ಯ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ, ಬೆಳಗಾವಿ

VISVESVARAYA TECHNOLOGICAL UNIVERSITY - BELAGAVI

GOVERNMENT ENGINEERING COLLEGE

RAMANAGARA - 562159

(Affiliated to Visvesvaraya Technological University)

DEPARTMENT

OF

ELECTRONICS & COMMUNICATION ENGINEERING

GEC Ramanagaram

तमसो मा ज्योतिर्गमय

LABORATORY MANUAL

EMBEDDED SYSTEM DESIGN (BEC601)

2025-26

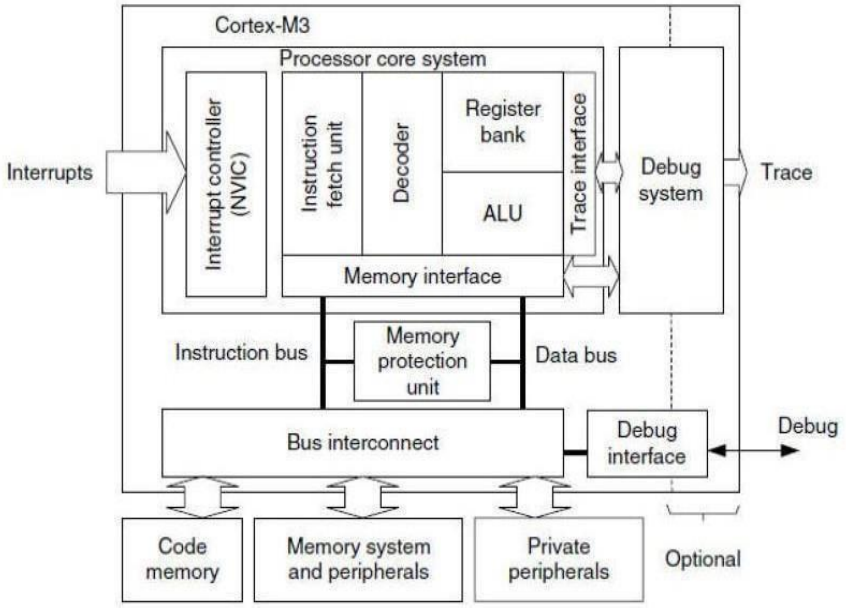
INTRODUCTION TO ARM CORTEX M3 PROCESSOR

1.1 Introduction

The ARM Cortex-M3 is a general purpose 32-bit microprocessor, which offers high performance and very low power consumption. The Cortex-M3 offers many new features, including a Thumb-2 instruction set, low interrupt latency, hardware divide, interruptible/continuable multiple load and store instructions, automatic state save and restore for interrupts, tightly integrated interrupt controller with Wake-up Interrupt Controller and multiple core buses capable of simultaneous accesses. Pipeline techniques are employed so that all parts of the processing and memory systems can operate continuously. Typically, while one instruction is being executed, its successor is being decoded, and a third instruction is being fetched from memory.

The processor has a Harvard architecture, which means that it has a separate instruction bus and data bus. This allows instructions and data accesses to take place at the same time, and as a result of this, the performance of the processor increases because data accesses do not affect the instruction pipeline. This feature results in multiple bus interfaces on Cortex-M3, each with optimized usage and the ability to be used simultaneously. However, the instruction and data buses share the same memory space (a unified memory system). In other words, you cannot get 8GBofmemoryspacejustbecauseyouhave separate bus interfaces.

A simplified block diagram of the Cortex-m3 architecture is shown below



It is worthwhile highlighting that the Cortex-M3 processor is not the first ARM processor to be used to create generic microcontrollers. The venerable ARM7 processor has been very successful in this market, The Cortex-M3 processor builds on the success of the ARM7 processor to deliver devices that are significantly easier to program and debug and yet deliver a higher processing capability.

2. INTRODUCTION TO ARM MICROCONTROLLER LPC1768

2.1 Architectural Overview

The LPC1768FBD100 is an ARM Cortex-M3 based micro controller for embedded applications requiring a high level of integration and low power dissipation. The ARM Cortex-M3 is a next generation core that offers system enhancements such as modernized debug features and a higher level of support block integration.LPC1768 operate upto 100MHz CPU frequency. It is the first series in the entire ARM Cortex-M line-up and is primarily targeted at 32-bit Microcontroller market. The ARM Cortex-M3 based MCUs have integrated many features that are once found only in high-end microprocessors.

ARM Cortex-M3 Core is designed and developed as a successor to the famous ARM-7 Architecture. It introduced several features like non-maskable interrupts, nested vector interrupts, atomic bit manipulation and memory protection. All these and many other features make the ARM Cortex-M3 Core a very famous and widely used ARM Architecture in the Microcontroller market. Some of the prominent vendors of ARM Cortex-M3 based MCUs are STMicroelectronics, NXP, Texas Instruments, Infineon, Microchip (Atmel), Toshiba, Analog Devices etc.

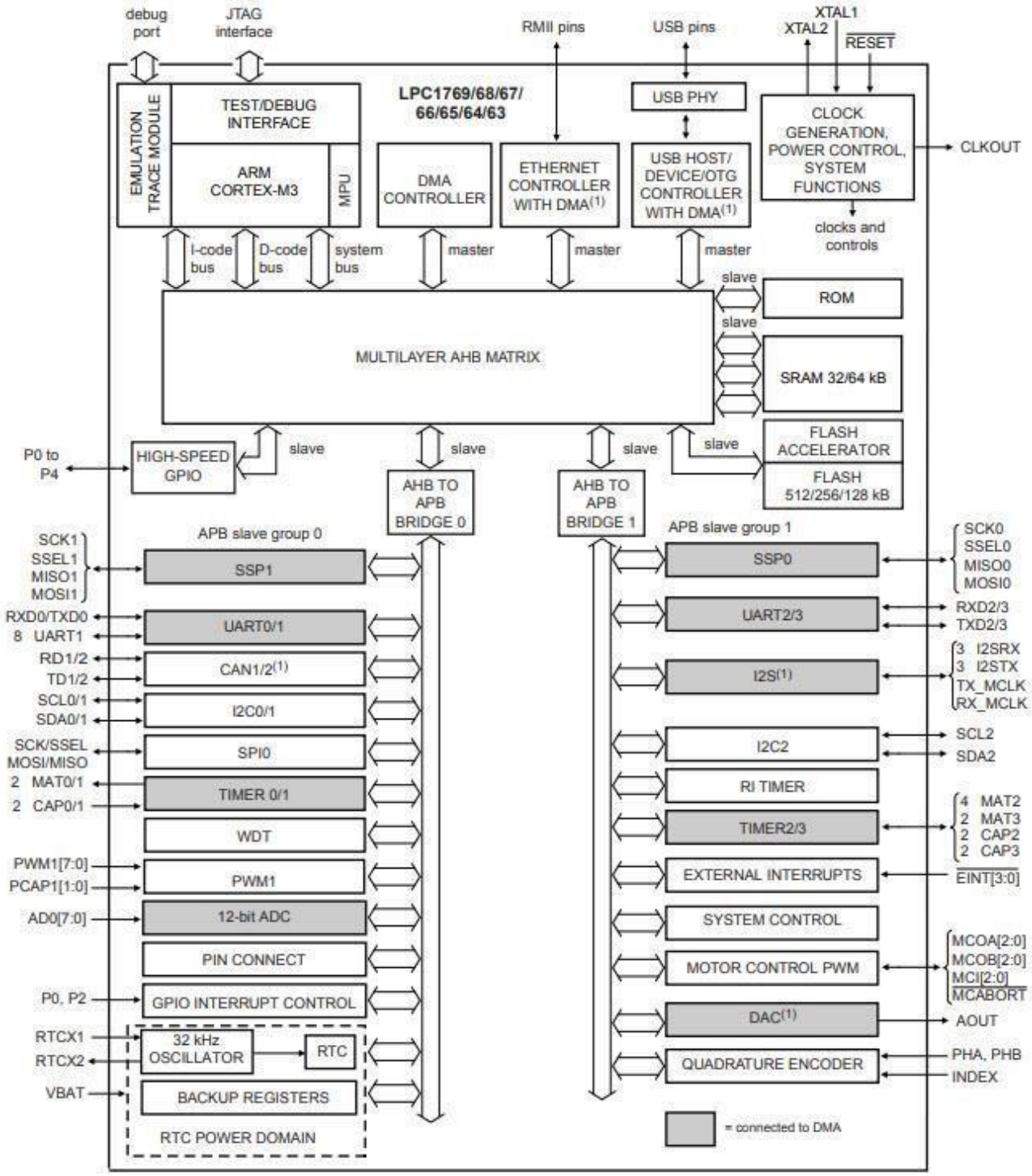
The LPC1768 MCU consists of the following peripherals: 512 kB Flash, 64 kB RAM, USB Interface,

Ethernet MAC, 4 UARTs, 2 CANs, 3 I2C Interfaces, SPI, 2 SSP Controllers, 2-input 2-output I2S, 8-channel ADC with 12-bit resolution, 10-bit DAC, 4 Timers, 8-channel DMA, 6-channel PWM, Motor Control PWM, low-power RTC, Quadrature Encoder and 70 GPIO Pins.

The following is a small list of the specifications of the LPC1768 MCU.

- ARM Cortex-M3 Processor with built-in NVIC and MPU and CPU Frequency of 100 MHz.
- On-chip Bootloader for in-system programming (ISP) and in-application programming (IAP).
- 512 kB of on-chip Flash and 64 kB of on-chip SRAM.
- 8-channel DMA Controller that can be used with ADC, DAC, Timers, UART, SSP, I2S.
- Multilayer AHB Matrix Interconnect with separate bus for AHB Master (CPU, DMA, Ethernet and USB). Split APB Bus for all other peripherals.
- Serial Interfaces
    - Ethernet MAC with RMII Interface
    - Full-speed USB 2.0 Controller
    - Four UARTs
    - 2-channel CAN Controller
    - 2 SSP Controllers
    - 3 I2C Interfaces
    - SPI
    - I2S
- Other Peripherals
    - 70 GPIO Pins
    - 8-channel 12-bit ADC
    - 10-bit DAC
    - 4-Timers
    - Dedicated PWM Block
    - Motor Control PWM
    - Quadrature Encoder Interface
    - RTC
    - WDT

2.2 BlockDiagramofLPC1768FBD10



2.3 List of the blocks:

- Nested vector interrupt controller
- General purpose DMA controller
- Fast general purpose parallel I/O
- USB interface
- 12-bitADC
- 10-bitDAC
- UART's
- SPI serial I/O controller
- SSP serial I/O controller
- I2C-bus serial I/O controllers
- General purpose 32-bit timers/external event counters
- Pulse width modulator
- Watch dog timer
- RTC and backup registers
- Clocking and Power Control
- System Control
- Reset
- Emulation and debugging

PART-A

Conduct the following Study experiments to learn ALP using ARM Cortex M3 Instruction set & Registers using an Keil uVision-4 tool/compiler software tool.

Experiment-1

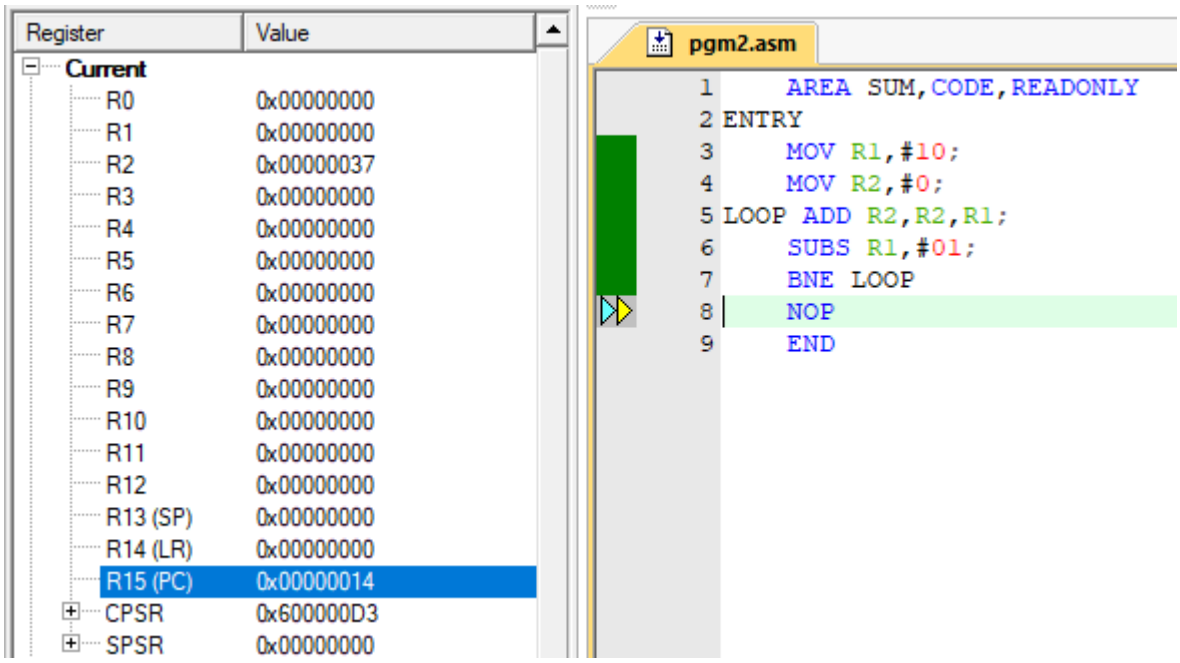
1. Write a program to find the sum of the first 10 integer numbers

Aim: To write an ALP to find the sum of first 10 integer numbers.

CODE:

```
AREA SUM,CODE,READONLY
        ENTRY
        MOV R1,#10;  Load 10 to register R1
        MOV R2,#0;  Empty the register R2 to store result
LOOP ADD R2,R2,R1; Add the content of R1 with result at R2
        SUBS R1,#01; Decrement R1 by 1
        BNE LOOP ; Repeat till R1=0
        NOP ;No operation
        END
```

Result:



Before Execution

R1=0X00000010H

R2=0X00000000H

After Execution

R1=0X00000000H

R2=0X00000037H

Outcome: After finishing this experiment the students are able to calculate the sum of the first 10 integer numbers using assembly language program.

Experiment-2

Write an Program to i) Multiply two 16-bit numbers. ii) Add two 32-bit numbers

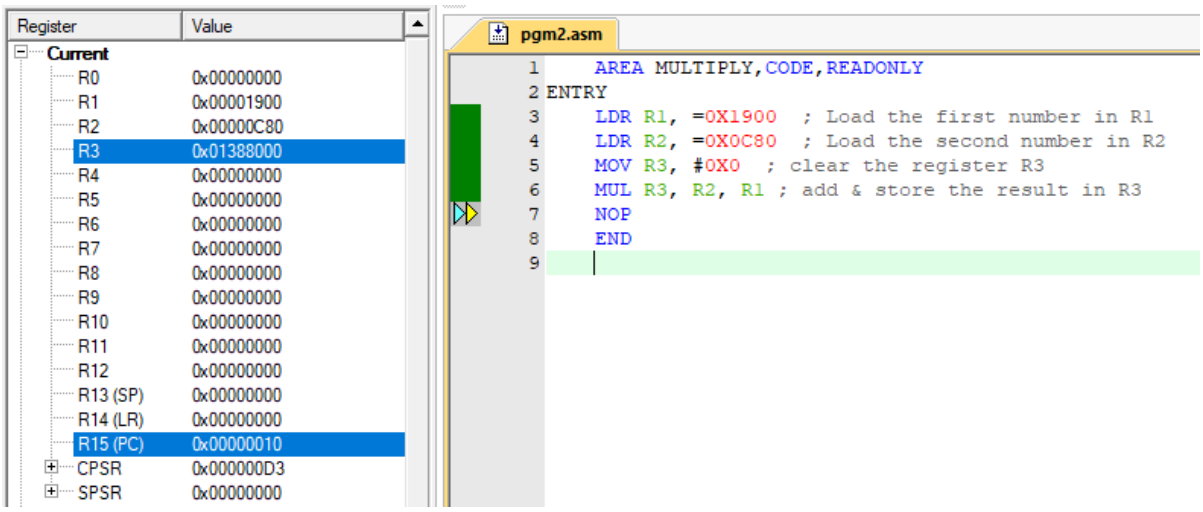
Aim: To write an ALP to multiply two 16-bit numbers and add two 32-bit numbers.

CODE:

i) Multiplication of two 16-bit numbers

```
        AREA MULTIPLY,CODE,READONLY
ENTRY
        LDR R1, =0X1900  ; Load the first number in R1
        LDR R2, =0X0C80  ; Load the second number in R2
        MOV R3, #0X0  ; clear the register R3
        MUL R3, R2, R1 ; multiply & store the result in R3
        NOP
        END
```

Result:



Before Execution

R1=0X00000000H

R2=0X00000000H

R3=0X00000000H

After Execution

R1=0X00001900H

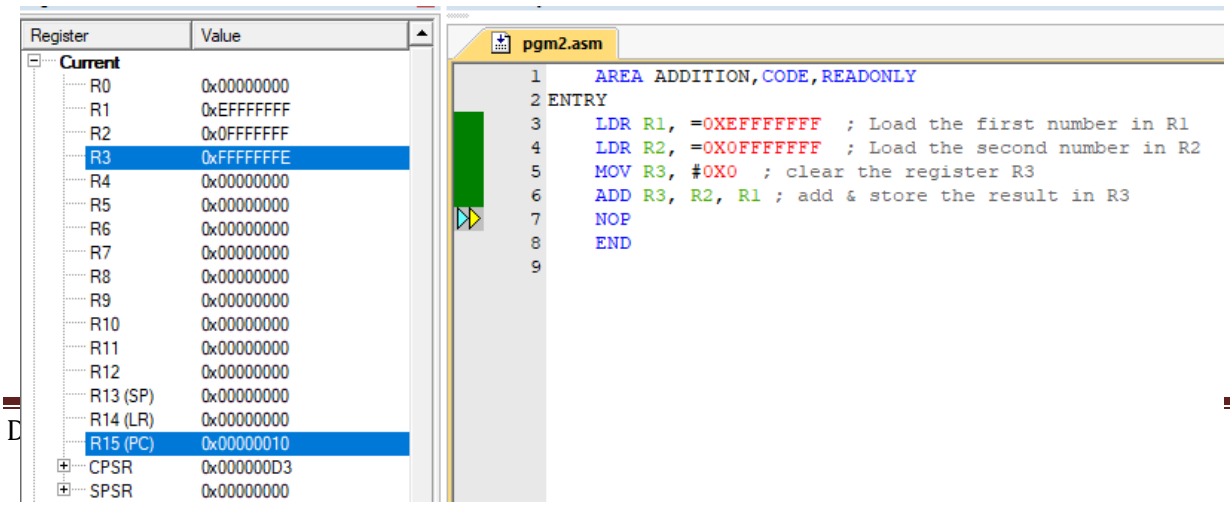
R2=0X00000C80H

R3=0X01388000H

ii) Addition of two 32-bit numbers

```
        AREA ADDITION,CODE,READONLY
ENTRY
        LDR R1, =0XEFFFFFFF  ; Load the first number in R1
        LDR R2, =0X0FFFFFFF  ; Load the second number in R2
        MOV R3, #0X0  ; clear the register R3
        ADD R3, R2, R1 ; add & store the result in R3
        NOP
        END
```

Result:



Before Execution

R1=0X00000000H

R2=0X00000000H

R3=0X00000000H

After Execution

R1=0XEFFFFFFFH

R2=0X0FFFFFFFH

R3=0XFFFFFFFEH

Outcome: After finishing this experiment the students are able to perform multiplication and addition of two numbers using assembly language program.

EXPERIMENT-3

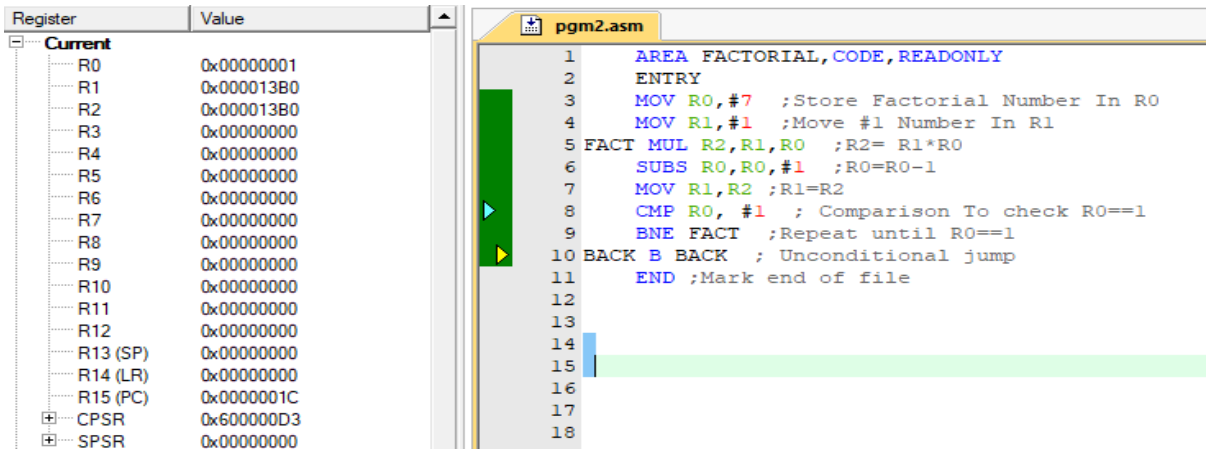
Write a program to find the factorial of a number

Aim: To write an ALP to find the factorial of a number

CODE:

```
AREA FACTORIAL,CODE,READONLY
        ENTRY
        MOV R0,#7  ;Store Factorial Number in R0
        MOV R1,#1  ;Move #1 Number in R1
FACT MUL R2,R1,R0  ;R2= R1*R0
        SUBS R0,R0,#1  ;R0=R0-1
        MOV R1,R2 ;R1=R2
        CMP R0, #1  ; Comparison to check R0==1
        BNE FACT  ; Repeat till R0=1
BACK B BACK  ; Unconditional jump
        END ;Mark end of file
```

Result:



Before Execution

R0=0X00000007H

R1=0X00000001H

R2=0X00000000H

After Execution

R0=0XEFFFFFFFH

R1=0X000013B0H

R2=0X000013B0H

Outcome: After finishing this experiment the students are able to find factorial of number using assembly language program.

EXPERIMENT-4

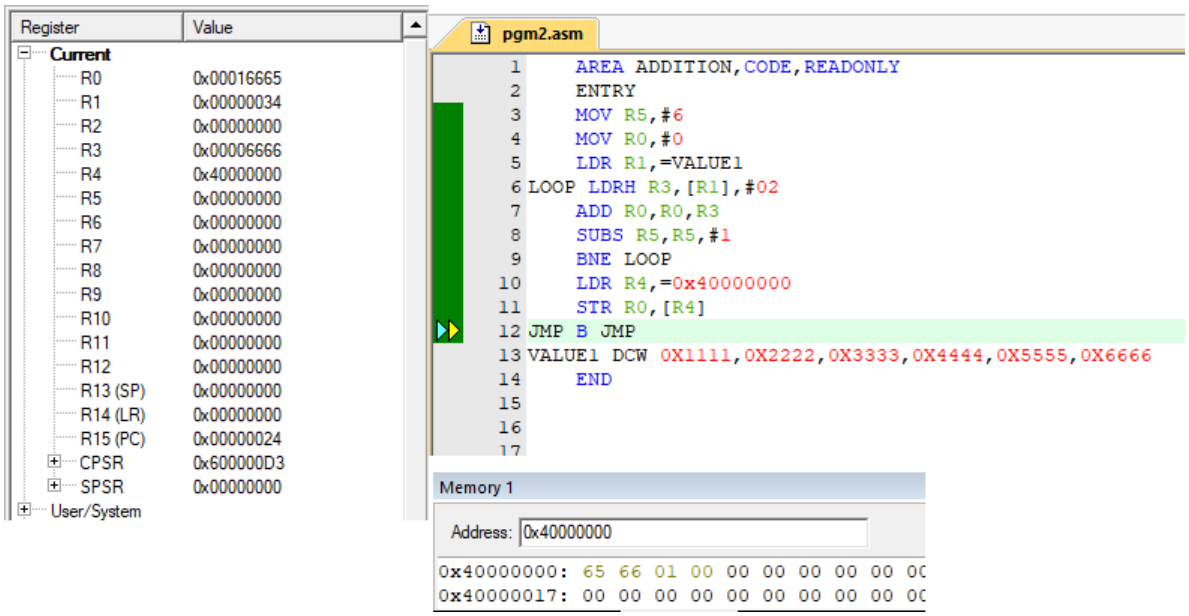
Write a program to add an array of 16 bit numbers and store the 32 bit result in internal RAM

Aim: To write an ALP to add an array of 16 bit numbers and store the 32 bit result in internal RAM.

CODE:

```
        AREA ADDITION,CODE,READONLY
        ENTRY
        MOV R5,#6; Initialise Counter To 6(i.e. N=6)
        MOV R0,#0 ; Initialise Sum to Zero
        LDR R1,=VALUE1; Loads the Address of First Value
LOOP LDRH R3,[R1],#02 ; Read 16 Bit Data
        ADD R0,R0,R3 ;Add R0=R0+R3
        SUBS R5,R5,#1 ; Decrement Counter
        BNE LOOP ;Look Back till Array Ends
        LDR R4,=0x40000000 ;Loads he Address Of Result
        STR R0,[R4];  stores the value of R0 into the memory location pointed to by R4
JMP B JMP; Unconditional jump
VALUE1 DCW 0X1111,0X2222,0X3333,0X4444,0X5555,0X6666 ; Array Of 16 Bit Numbers(N=6)
        END
```

Result:



Before Execution

R0=0X00000000H

R5=0X00000006H

After Execution

R0=0X00016665H

R5=0X00000000H

Outcome: After finishing this experiment the students are able to add an array of 16 bit numbers and store the 32 bit result in internal RAM using assembly language program.

EXPERIMENT-5

Write a program to find the square of a number (1 to 10) using a look-up table

Aim: To write an ALP to find the square of a number (1 to 10) using a look-up table

CODE:

```
AREA SQUARE,CODE,READONLY
        ENTRY
        LDR R0, =VALUE1; Load start address of Lookup table
        MOV R1,#0X0003; Load no whose square is to be find
        MOV R1,R1,LSL#0x2; Generate address corresponding to square of given no
        ADD R0,R0, R1; Load address of element in Lookup table
        LDR R3,[R0] ;Get square of given no in R3
BACK B BACK
VALUE1 DCD 0X0000,0X0001,0X0004,0X0009,0X0010, 0X0019,0X0024,0X0031,
0X0040,0X0051,0X0064 ; ;Lookup table contains Squares of no's from 0 to 10 (in hex)
        END
```

Result:

Before Execution

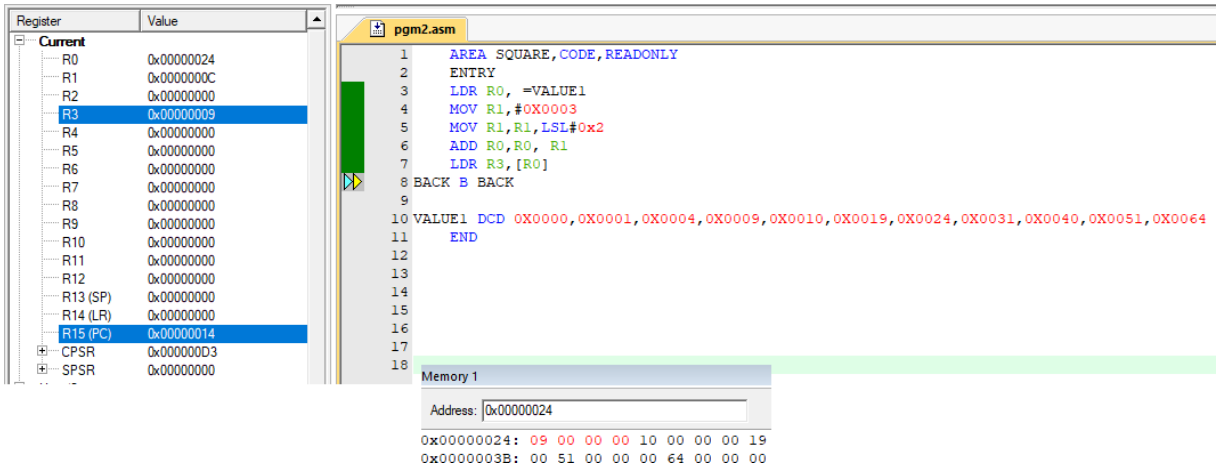
R1=0X00000003H

R3=0X00000000H

After Execution

R1=0X00000000CH

R3=0X00000009H



Outcome: After finishing this experiment the students are able to find the square of a number (1 to 10) using a look-up table by using assembly language program.

Experiment-6

Write a program to find the largest and smallest number in an array of 32 numbers

Aim: To write an ALP to find the largest and smallest number in an array of 32 numbers

CODE:

```
        AREA LARGEST,CODE,READONLY
        ENTRY
        MOV R5,#4  ; INTIALISE COUNTERTO4(i.e. N=5)
        LDR R1,=VALUE1 ; LOADS THE ADDRESS OF FIRST VALUE
        LDR R2,[R1],#4 ; WORD ALIGN T0 ARRAY ELEMENT
LOOP LDR R4,[R1],#4 ; WORD ALIGN T0 ARRAY ELEMENT
        CMP R2,R4 ; COMPARE NUMBERS
        MOVCC R2,R4 ; MOVLO/MOVCC/MOVLS for largest and
MOVHS/MOVCS/MOVHI for smallest
        SUBS R5,R5,#1 ; DECREMENT COUNTER
        BNE LOOP ; LOOP BACK TILL ARRAY ENDS
BACK B BACK
VALUE1 DCD 0X44444444,0X22222222,0X11111111,0X33333333,0XAAAAAAAA
        END
```

Result:

Before Execution

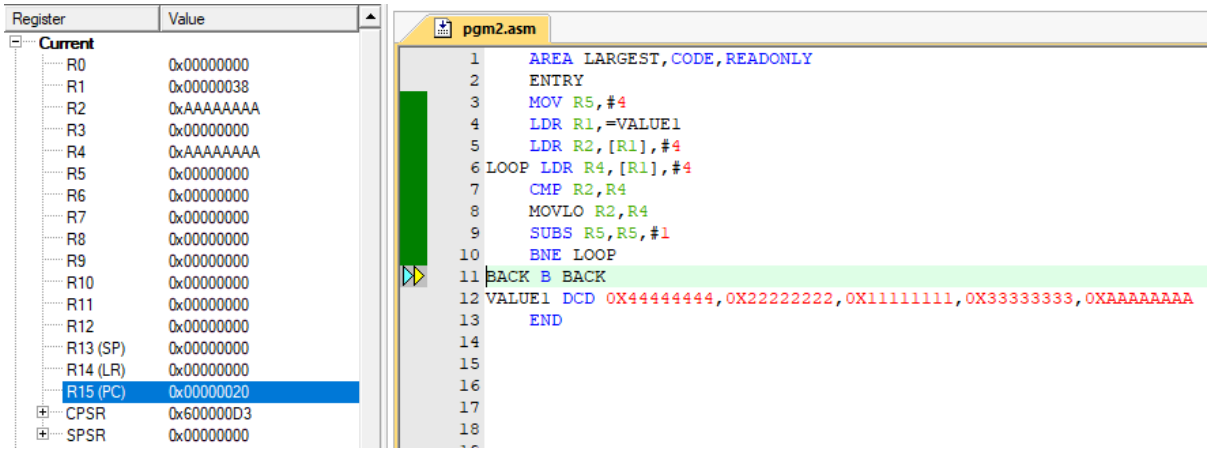
R2=0X00000000H

R5=0X00000004H

After Execution

R2=0XAAAAAAAAH

R5=0X00000000H



Before Execution

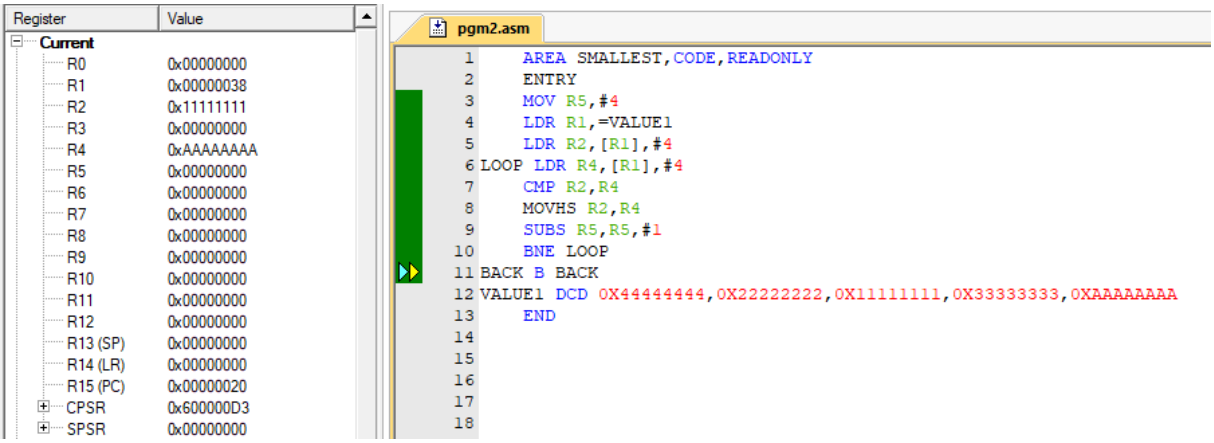
R2=0X00000000H

R5=0X00000004H

After Execution

R2=0X11111111H

R5=0X00000000H



Outcome: After finishing this experiment the students are able to find the largest or smallest number in an array of 32 numbers by using assembly language program.

Experiment-7

Write a program to arrange a series of 32 bit numbers in ascending/descending order

Aim: To write an ALP to find the Ascending and Descending number in an array of 32 numbers

CODE:

```
AREA ASCENDING,CODE,READONLY
        ENTRY ;Mark first instruction to execute
        MOV R8,#04 ;Initialize Counter To 4( number of elements)
AGAIN LDR R0,=0X40000000 ;Loads The Address To Register Ro
        MOV R5,#03 ;Number of pass
LOOP  LDR R2,[R0],#04 ;Load First Number
        LDR R3,[R0] ;Load Second Number
        CMP R2,R3 ;Compare Numbers
        BLT LOOP2 ;If The First Number Is<Then Goto Loop2
        STR R2,[R0],#-04 ;Interchange Number R2&R3
        STR R3,[R0]
        ADD R0,#04 ;Restore  The PTR
LOOP2          SUBS  R5,R5,#1 ;Decrement Counter
        CMP R5,#0 ;Compare Counter To 0
        BNE LOOP ;Loop Back Till Array Ends
        SUBS R8,R8,#1 ;Decrement Counter
        CMP R8,#0 ;Compare Counter To 0
        BNE AGAIN ;Loop Back Till Array Ends
STOP B STOP
        END
```

Result:

Before Execution

Memory Window-1 0X40000000:22222222 11111111 77777777 55555555

After Execution

MemoryWindow-1 0X40000000:11111111 22222222 55555555 77777777

CODE:

```
AREA DESCENDING,CODE,READONLY
        ENTRY ;Mark first instruction to execute
        MOV R8,#04 ;Initialize Counter To 4( number of elements)
AGAIN LDR R0,=0X40000000 ;Loads The Address To Register Ro
        MOV R5,#03 ;Number of pass
LOOP  LDR R2,[R0],#04 ;Load First Number
        LDR R3,[R0] ;Load Second Number
        CMP R2,R3 ;Compare Numbers
        BGT LOOP2 ;If The First Number Is> Then Goto Loop2
        STR R2,[R0],#-04 ;Interchange Number R2&R3
        STR R3,[R0]
        ADD R0,#04 ;Restore  The PTR
LOOP2          SUBS  R5,R5,#1 ;Decrement Counter
        CMP R5,#0 ;Compare Counter To 0
        BNE LOOP ;Loop Back Till Array Ends
        SUBS R8,R8,#1 ;Decrement Counter
        CMP R8,#0 ;Compare Counter To 0
        BNE AGAIN ;Loop Back Till Array Ends
STOP B STOP
        END
```

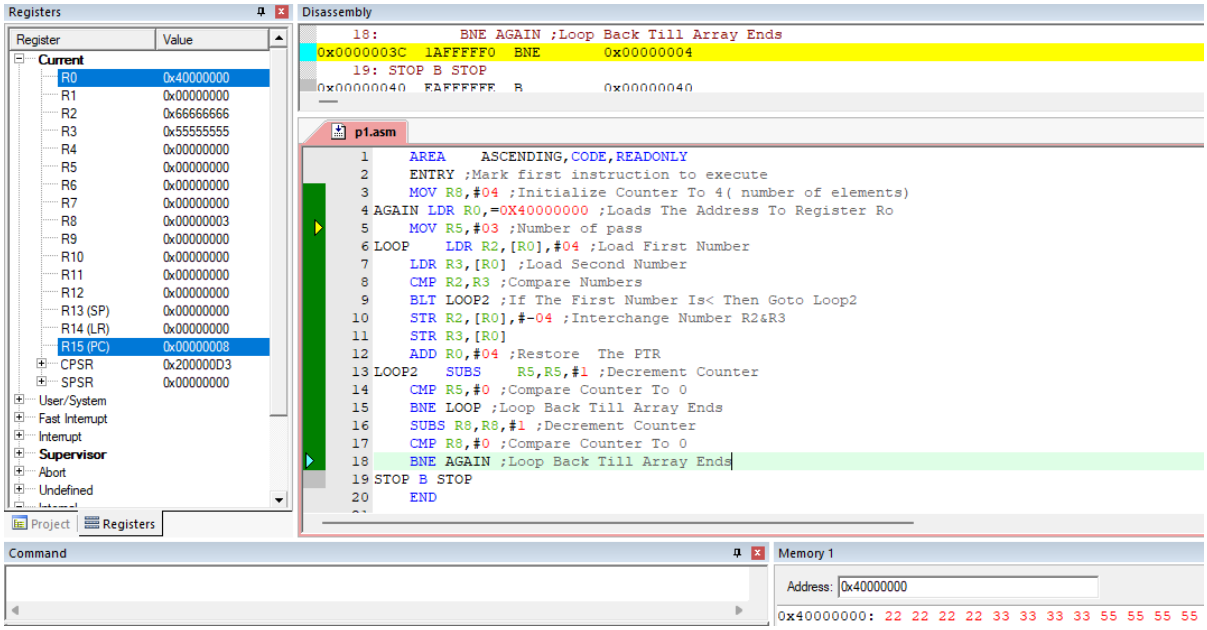
Result:

Before Execution

Memory Window-1 0X40000000:22222222 11111111 77777777 55555555

After Execution

Memory Window-1 0X40000000:77777777 55555555 22222222 11111111



Outcome: After finishing this experiment the students are able to arrange a series of 32 bit numbers in ascending/descending order by using assembly language program.

Experiment-8

Write a program to count the number of ones and zeros in two consecutive memory locations

Aim: To write an ALP to count the number of ones and zeros in two consecutive memory locations

CODE:

```
        AREA ONEZERO,CODE,READONLY
        ENTRY ;Mark first instruction to execute
        MOV R2,#0 ;Counter For Ones
        MOV R3,#0 ;Counter For Zeros
        MOV R7,#2 ;Counter To Get Two Words
        LDR R6,=VALUE ;Loads The Address Of Value
LOOP MOV R1,#32 ;32 Bits Counter
        LDR R0,[R6],#4 ;Get The 32 Bit Value
LOOP0 MOVS R0,R0,ROR#1 ;Right Shift To Check Carry Bit(1's/0's)
        BHI ONES ;If Carry Bit Is 1 Goto Ones Branch Otherwise Next
ZEROS ADD R3,R3,#1 ;If Carry Bit Is 0 Then Increment The Counter By 1(R3)
        B LOOP1 ;Branch To Loop1
ONES ADD R2,R2,#1 ;If Carry Bit Is 1 Then Increment The Counter By 1(R2)
LOOP1 SUBS R1,R1,#1 ;Counter Value Decremented By 1
        BNE LOOP0 ;If Not Equal Goto To Loop0 Checks 32bit
        SUBS R7,R7,#1 ;Counter Value Decremented By 1
        CMP R7,#0 ;Compare Counter R7 To 0
        BNE LOOP ;If Not Equal Goto To Loop
BACK B BACK
VALUE DCD 0X00000003,0X00000002;Two Values In An array
 END ;Mark end of file
```

Result:

Before Execution

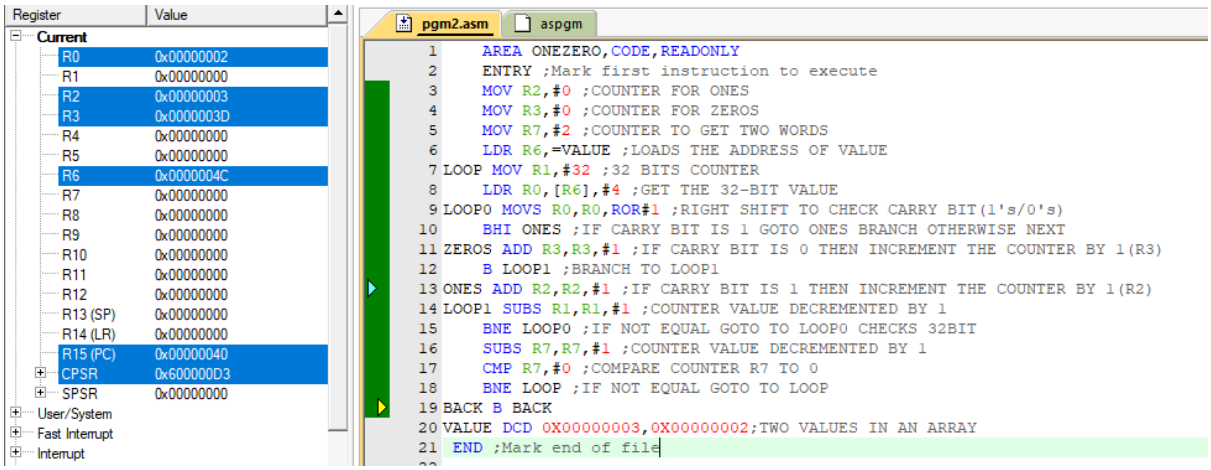
R2=0X00000000H

R3=0X00000000H

After Execution

R2=0X00000003H

R3=0X0000003DH



Outcome: After finishing this experiment the students are able to count the number of ones and zeros in two consecutive memory locations by using assembly language program.

PART-B

Conduct the following experiments on an ARM Cortex M3 using evaluation board using evaluation version of Embedded 'C' & Keil uVision-4 tool/compiler.

Experiment-9

Display the Hex digits 0 to F on a 7-segment LED interface, with a suitable delay in between

Aim: To Interface 7-segment LED to display the Hex digits 0 to F using ARM CM3(LPC1768).

Code:

```
#include <LPC17xx.h>
unsigned int delay, count=0, Switchcount=0,j;
unsigned int Disp[16]={0x000003f0, 0x00000060, 0x000005b0, 0x000004f0,
0x00000660,0x000006d0,0x000007d0, 0x00000070, 0x000007f0, 0x000006f0,
0x00000770,0x000007c0,0x00000390, 0x000005e0, 0x00000790, 0x00000710 };
#define ALLDISP  0x00180000        //Select all display
#define DATAPORT 0x00000ff0        //P0.4 to P0.11 : Data lines connected to drive Seven
Segments
int main (void)
{
        LPC_PINCON->PINSEL0 = 0x00000000;//   P0.0 to P0.15
        LPC_PINCON->PINSEL1 = 0x00000000; P0.16 to P0.31
        LPC_GPIO0->FIODIR =0x00180ff0; // Set pins as outputs
        while(1)
        {
                LPC_GPIO0->FIOSET |= ALLDISP; //Set pins high
                LPC_GPIO0->FIOCLR =0x00000ff0; // clear the data lines to 7-segment
displays
                LPC_GPIO0->FIOSET = Disp[Switchcount];   // get the 7-segment display
value from the array
                        for(j=0;j<3;j++)
                        for(delay=0;delay<30000;delay++);     // 1s delay
                        Switchcount++;
                        if(Switchcount == 0x10)// 0 to F has been displayed ? go back to 0
```

```
 {
                                Switchcount = 0;
                                LPC_GPIO0->FIOCLR  =  0x00180ff0;
                                 }
        }
}
```

Result: The 7 segment LED is interfaced with LPC 1768 to display Hex digits 0 to F.

Outcome: After finishing this experiment the students are able to write an embedded C Program to display 0-F Hex Digits by interfacing 7-segment LED to the Port pins of ARM CM3(LPC1768)

Experiment-10

Interface a simple Switch and display its status through Relay, Buzzer and LED

Aim: To Interface Simple Switch and display its status through Relay Buzzer and LED using ARM CM3(LPC1768).

CODE:

```
#include <LPC17xx.H>
int main(void)
{
        unsigned int i;
        LPC_PINCON->PINSEL1 &= 0x00000000; //P0.16 to P0.31 as GPIO(P0.24 and
P0.25
        LPC_GPIO0->FIODIR |= 0x03000000;  //P0.24 configured output for Buzzer and
P0.25 configured output for LED/Relay
  while(1)
  {
     if(!(LPC_GPIO2->FIOPIN & 0x00000800)) // Is GP_SW(SW4,P2.11 is Pressed??
       {
        for(i=0;i<10;i++)
        {
         LPC_GPIO0->FIOSET = 0x03000000;      //relay on
                for(i=0;i<10000;i++);
                }
                  }
        else
        {
        LPC_GPIO0->FIOCLR = 0x03000000;       //relay off
        for(i=0;i<100000;i++);
            }
```

```
        }
}       //end int main(void)
```

Result: The simple Switch is interfaced and displayed its status through Relay Buzzer and LED with LPC 1768.

Outcome: After finishing this experiment the students are able to write an embedded C Program to interface a simple switch and to turn on the buzzer relay and LED using ARM CM3(LPC1768)

Experiment-11

Interface a Stepper motor and rotate it in clockwise and anti-clockwise direction.

Aim: To interface a Stepper motor and rotate it in clockwise and anti-clockwise direction using ARM CM3(LPC1768)

Code:

```
#include <LPC17xx.H>
void clock_wise(void);
void anti_clock_wise(void);
unsigned long int var1;// for holding a step changing value
unsigned int i=0,j=0,k=0;//used in for loop
int main(void)
{
	LPC_PINCON->PINSEL4 = 0x00000000;  //P2.0 to P2.15 as a GPIO(P2.0 to P2.3)(PIN-75,74,73,70)
	LPC_GPIO2->FIODIR = 0x0000000F;		//0bit-P2.0,1bit-P2.1,2bit-P2.2,3bit-P2.3 output port
	while(1)
	{
		for(j=0;j<50;j++)		//50 times in Clock wise Rotation
			clock_wise();

		for(k=0;k<65000;k++);		//Delay to show  anti_clock Rotation

		for(j=0;j<50;j++)			//50 times in  Anti Clock wise Rotation
			anti_clock_wise();

		for(k=0;k<65000;k++);		//Delay to show clock Rotation
```

```
}
//End of while(1)
}        //End of main
void clock_wise(void)
{
        var1 = 0x00000001;                          //For Clockwise
   for(i=0;i<=3;i++)                          //for A B C D Stepping
        {
          LPC_GPIO2->FIOCLR =  0X0000000F;
          LPC_GPIO2->FIOSET    =    var1;//0x01-step    A(2.0),0x02-step    B(2.1),0x04-step
C(P2.2),0X01-step D(P2.3)
                var1 = var1<<1;         //Shift var1 to left by one bit For Clockwise
      for(k=0;k<3000;k++);                    //for step speed variation
   }
}
void anti_clock_wise(void)
{
        var1 = 0x0000008;                           //For Anticlockwise
   for(i=0;i<=3;i++)                          //for A B C D Stepping
   {
           LPC_GPIO2->FIOCLR =  0X0000000F;
                LPC_GPIO2->FIOSET =  var1;//0x08-step D(2.3),0x04-step C(2.2),0x02-step
B(P2.1),0X08-step A(P2.0)
                var1 = var1>>1;                 //Shift var1 to right by one bit For Anticlockwise
      for(k=0;k<3000;k++);                    //for step speed variation
   }
}
```

Result: The stepper Motor is interfaced with LPC 1768 and it is rotated in clockwise and anticlockwise direction for 50 times successfully.

Outcome: After finishing this experiment the students are able to write an embedded C Program to interface a stepper Motor and rotate it in clockwise and anti-clockwise direction using ARM CM3(LPC1768).

Experiment-12

Interface a DAC and generate Triangular and Square waveforms

Aim: To interface DAC and generate Triangular and Square waveforms using ARM CM3(LPC1768)

Code:

```
#include <LPC17xx.H>
int main()
{
Unsigned long int temp=0x00000000;
Unsigned int i=0;
LPC_PINCON->PINSEL0&=0xFF0000FF; //Configure P0.4 to P0.11 as GPIO
LPC_GPIO0->FIODIR|=0x00000FF0;
while(1)
{
//output 0 to FE
for(i=0;i!=0xFF;i++)
{
temp=i;
temp=temp<<4;
LPC_GPIO0->FIOPIN=temp;
}
//output FF to 1
for(i=0xFF;i!=0;i--)
{
temp=i;
temp=temp<<4;
LPC_GPIO0->FIOPIN=temp;
}
}//End of while(1)
```



```
}//End of main()
```

SQUARE WAVEFORM

```
#include <LPC17xx.H>
Void delay(void);
int main()
{
LPC_PINCON->PINSEL0&=0xFF0000FF; // Configure P0.4 to P0.11 as GPIO
LPC_GPIO0->FIODIR|=0x00000FF0;
while(1)
{
LPC_GPIO0->FIOPIN=0x00000FF0;
delay();
LPC_GPIO0->FIOCLR=0x00000FF0;
delay();
}
}
Void delay(void)
{
}
Unsigned int i=0;
for(i=0;i<=9500;i++);
}
```

Result: The DAC is interfaced with LPC 1768 and triangular & Square waveform is generated and observed using CRO.

Outcome: After finishing this experiment the students are able to write an embedded C Program to interface DAC and triangular & Square waveform using ARM CM3(LPC1768).